

第 5 课 · 评测与可观测

怎么证明你改完变好了

第 5 课 · 120 分钟（讲授 45 + 实操 65 + 小面试 10）

一、这节课的目标

三件事：第一，理解为什么「准确率」一个数字是不够的；第二，学会读混淆矩阵找到该改什么；第三，把 trace 和成本接进评测，让系统可解释、可核算。

二、口语化讲稿

先说一个数字，然后质疑它

我先把我们项目当前的评测结果放出来。200 条用例，准确率 100%。

```
cases evaluated      200 / 200
exact accuracy       100.0% (200 correct)
  by difficulty      easy:100%(n=68)  medium:100%(n=66)  hard:100%(n=66)

abstentions          33
  abstention precision 100.0% (33/33 refusals were correct refusals)
  unknown-case recall  100.0% (33/33 unknowable cases correctly refused)
false-confidence rate 0.0% (0 wrong answers given at >= 70% confidence)
answer parse rate     100.0% (200/200 parseable)
cost                  $0.3115 total, $0.001558 per diagnosis
latency               p50 4ms  p90 5ms  p99 7ms
```

现在，**请带着怀疑看这个 100%**。一个好数字如果解释不了，它就是个坏数字。所以我要求你们做的第一件事是：找出它为什么这么高。

答案是：**数据集和评分器出自同一份剧本**。同一份 faults.yaml 既定义了生成器输出的签名，也定义了评分器要匹配的模式。换句话说，离线推理器是在一个**闭环**里被评分的。

这个数字**不能**外推成「真实模型在真实数据上会怎样」。想验证这一点，把 AILAB_LLM_BACKEND 指向一个真实服务再跑一遍。

那这个评测还有价值吗？有。两件事让它依然可信，而且这两件事是我们**刻意设计**的。

第一：语料是故意带噪声的

如果每条失败 job 的日志里只出现它自己那个故障的字符串，那这个任务的本质就是「子串匹配」——看一眼就答对了，没有区分度。

所以我们往**健康的和无关的日志行**里掺了别的故障的告警：一次短暂的 allreduce 重试、一次短暂的 DNS 抖动、一次预取队列触顶。大约 28% 的失败 job 带着这类噪声。它们会匹配到对手场景的模式。

datagen/world.py。这些行属于**别的**故障，但会出现在健康运行的日志里。

```
AMBIENT_WARN = [
    ...
    "rank {rank} slow allreduce: 1 collective took {st}s, retrying",
    "transient NCCL retry on rank {rank} succeeded after 1 attempt",
    "temporary failure in name resolution for shard {shard}, retrying",
    "prefetch queue hit cache limit {qmax} on rank {rank}, ...",
]
```

那噪声为什么没有把结论搅乱？因为模式**权重的设计起 IDF 的作用**：通用告警权重 0.2，因果行权重 1.0。这和 BM25 里 IDF 降低常见词权重的思路是同一个。

第二：校准指标造不了假

这是我觉得这个评测里最有意思的部分。看报告里这两项。

指标	含义	为什么它重要
abstention precision	拒答的案例里，有多少是**本来就该拒**的	一个全部拒答的系统，这个值是 100%，但准确率只有 6%

指标	含义	为什么它重要
false -con fide nce rate	错了但还 很自信的 占比	accuracy 完全掩盖 这一点

把两项一起看，你就没法作弊了。全部拒答：准确率崩，拒答精确率满分。从不拒答：准确率好看，拒答精确率 0。两个数字必须一起报，权衡才会暴露出来。

假自信率是另一个维度。一个以 0.97 置信度给出的错误答案，会让工程师付出真实的时间成本——他去查了一条错误的路。准确率不区分「错了」和「错了还很自信」，这个指标区分。

混淆矩阵：告诉你去改哪里

当准确率不理想时，你需要的不是「再调调提示词」，而是具体在混淆哪两类。混淆矩阵就是干这个的。它列出「预测成什么 ← 实际是什么」，按次数排序。

开发早期的一个真实快照。一眼看出问题：系统在证据不足时硬猜。

```
most frequent confusions (predicted <- truth):
  watchdog_hang      <- insufficient_evidence  x8
  node_evicted       <- insufficient_evidence  x6
  process_killed_sigkill <- insufficient_evidence  x4
```

这是我早期跑出来的真实快照。看第一列全是同一个东西——系统在证据不足的时候胡乱猜了一个根因。这时候你该改什么？不是调提示词，是要给打分逻辑加一个「拒答」的出口。这就是混淆矩阵的价值：它把「系统不好」翻译成了「具体哪个逻辑缺失」。

这个快照来自这个项目开发过程中的一次真实评测。修完之后，unknown 类的准确率从 0% 升到 100%，整体准确率从 77.5% 升到 98% 以上。复现方法：`signals.py` 里一共有 7 处 `return Verdict(INSUFFICIENT\_EVIDENCE, ...)` 的守卫（阈值不足、证据两可、级联无独有签名、退出码单独不足以定因、遥测缺失等）。把它们全部禁用，未知类准确率立刻从 100% 掉到 0%，整体准确率掉到 81.7%。这正好说明：拒答不是找不到答案，是一组刻意设计的守卫。

禁用的守卫	unknown 准确率	整体准确率
(不 disable, 正常状态)	100%	100%
只禁用 1 条	100%	100%
全部 7 条都禁用	0%	81.7%

注意第二行——**只禁用一条，什么都不会变**。这说明这些守卫不是「一条规则」，而是纵深防御：同一个 case 会有好几条规则同时拦它。这是刻意的冗余，因为任何单独一条都可能被某个新场景绕过。

可观测：没有 trace 就没法调试

最后讲可观测。它的价值不是「好看」，而是回答几个具体问题：这次请求为什么慢？模型实际收到了什么？某个租户此刻花了多少钱？答案为什么变了？

问题	靠什么回答
这次请求为什么慢	每一步一个 span，带耗时
答案为什么变了	工具调用轨迹——等价于数据库的执行计划
这个租户花了多少	按租户记账的 token 与美元

问题	靠什么回答
模型收到了什么	可选的内容捕获（默认关，因为又大又敏感）

第三个问题直接接到第 4 课——成本预算就是从这套记账里读的。所以可观测不只是「看」，它同时是**控制的输入**。

第四个我说一下为什么默认关闭。把完整的提示词和响应存下来，体积很大而且可能含敏感内容。**默认关、需要时开**，比默认开、出事再删要安全得多。

三、动手

实操 1：跑一次完整评测，读报告

```
make eval
ailab-ops eval --difficulty hard          # 00 hard
ailab-ops eval --limit 500 --out runs/r1.json
```

重点看三处：by category（哪一类失败最多）、by difficulty（如果 easy 高而 hard 低，说明系统在模式匹配而不是推理）、以及最下面的「how to read this」。

实操 2：亲手把校准指标调坏

改 src/ailab_ops/signals.py 里的 MIN_MARGIN，重跑 make eval。

```
PYTHONPATH=src python3 -c "
import ailab_ops.signals as S
print('0000:', S.MIN_MARGIN)
# 0 MIN_MARGIN 0000 00000 eval
# 00000000000000000000 unknown recall 00000
"
```

这个实验的目的是让你**亲眼看到权衡在动**。把阈值调激进（更容易下结论），准确率会掉、假自信率会升；调保守，未知案例召回会升、但可能在简单案例上无谓地拒答。没有免费的午餐，你要选一个点。

实操 3：看一次完整 trace

```
PYTHONPATH=src python3 -c "
from ailab_ops.runtime import build_runtime
```

```
rt = build_runtime()
job = rt.world.failed_jobs()[0]
res, tr = rt.diagnose(f'Why did {job.job_id} fail?')
import json
print(json.dumps(tr.trace.to_dict(), ensure_ascii=False, indent=2)[:1500])
"
```

四、小面试（10 分钟）

Q1: 怎么给一个没有标准答案的 agent 定义「好」？

这是本课最核心的题。分几层答：

1. ****先分类****：哪些是可判定的（有标准答案，比如根因是什么），哪些是主观的（回答得好不好）。前者用准确率，后者才需要 LLM-as-judge。
2. ****能不用 judge 就不用****。judge 本身会错、会偏、成本高。本项目的诊断任务有 ground truth，所以直接算准确率，比请另一个模型来打分可靠得多。
3. ****必须用 judge 时****：固定 judge 模型和版本、给出评分细则、在人工标注过的小集上校准 judge 本身。
4. ****永远报一组指标而不是一个****：准确率 + 校准 + 成本 + 延迟。

「能不用 judge 就不用」是加分项。很多人一上来就说 LLM-as-judge，说明没意识到 judge 本身是个需要评测的组件。

Q2: 你怎么知道改完 prompt 之后是变好还是变坏？

答法要具体：「我会先固定一个 golden set，跑之前和之后的评测，对比准确率和混淆矩阵。如果只看几个 case 感觉变好了，那是不可信的——很可能只是那几个 case 变好，别的变差了。」

- golden set 要分层采样，不能全是简单案例
- 要包含反例（正确答案是拒答的案例）
- 对比要看混淆矩阵的变化，不只看总准确率
- 评测必须可复跑——所以要把用例清单和配置一起存下来

最后一条容易被忽略。本项目的 `write\_cases` 会把用例清单和报告一起落盘，因为**没有用例清单，报告就不可复跑、不可比对**。

Q3: 线上没有 ground truth, 怎么监控质量?

- 代理指标: 工具调用成功率、答案解析成功率、拒答率、平均步数
- 异常检测: 拒答率突然飙升、步数突然变多, 通常意味着上游数据或模型变了
- 抽样人工标注: 定期抽一批线上 case 人工判, 用来校准代理指标
- 用户反馈信号: 有没有人追问「你确定吗」、有没有人换个说法再问一遍

Q4: 成本和延迟怎么权衡?

标准答法: 「这两者通常是**同一个方向**的——步数多、上下文长, 同时让成本和延迟都上升。所以降低步数预算是双赢的。真正的权衡在**质量 vs 成本**: 降级到只读证据的路径很便宜很快, 但答案质量低。所以我们的做法是分场景——交互式的走完整诊断, 批量的可以走降级路径, 并且明确标注。」

五、作业

1. 把 `MIN\_MARGIN` 从 0.18 改成 0.05 和 0.50, 各跑一次 `make eval`, 记录准确率、拒答精确率、假自信率三个数字的变化, 画成一张表。
2. 给自己找到的混淆对写一个修复方案。比如如果系统总是把 `memory\_leak` 误判成 `oom\_host`, 你会改什么?
3. 打开 `data/generated/ground\_truth.jsonl`, 找出所有 `insufficient\_evidence` 的记录, 用 `ailab-ops inspect` 看它们的日志——确认它们**真的**没有决定性证据。